complete Terraform setup Akbar Sir can use to:

- Create a **VPC**
- Create **7 subnets** (6 private, 1 public)
- Create **2 security groups** (allow SSH & HTTP)
- Launch **1 EC2 instance** in the public subnet
- Use a **user data shell script** to install Apache (`httpd`)
- All **variables** are separated in a `variables.tf` file

✅ Folder Structure:

css
CopyEdit
```
cloud-vpc-project/
├── main.tf
├── variables.tf
├── terraform.tfvars
├── userdata.sh
```

🔹 1. `variables.tf`

hcl
CopyEdit
```
variable "region" {
  default = "us-east-1"
}
```

```
variable "vpc_cidr" {
  default = "10.0.0.0/16"
}

variable "public_subnet_cidr" {
  default = "10.0.1.0/24"
}

variable "private_subnet_cidrs" {
  type    = list(string)
  default = ["10.0.2.0/24", "10.0.3.0/24", "10.0.4.0/24",
"10.0.5.0/24", "10.0.6.0/24", "10.0.7.0/24"]
}

variable "instance_type" {
  default = "t2.micro"
}

variable "ami_id" {
  description = "Amazon Linux 2 AMI"
  default     = "ami-0c55b159cbfafe1f0" # Change for your region
}
```

🔹 2. `terraform.tfvars`

h
CopyEdit

```
region                = "us-east-1"
instance_type         = "t2.micro"
ami_id                = "ami-0c55b159cbfafe1f0"
```

🔹 3. `userdata.sh`

bash
CopyEdit

```
#!/bin/bash
yum update -y
yum install -y httpd
echo "<h1>Welcome to Cloud Infotech - Web Server Launched by Akbar
Sir</h1>" > /var/www/html/index.html
systemctl start httpd
systemctl enable httpd
```

🔹 4. `main.tf`

hcl
CopyEdit

```
provider "aws" {
  region = var.region
}

# VPC
resource "aws_vpc" "main_vpc" {
  cidr_block = var.vpc_cidr
  tags = {
    Name = "akbar-vpc"
  }
}

# Internet Gateway
resource "aws_internet_gateway" "gw" {
  vpc_id = aws_vpc.main_vpc.id
```

```
}

# Public Subnet
resource "aws_subnet" "public_subnet" {
  vpc_id                  = aws_vpc.main_vpc.id
  cidr_block              = var.public_subnet_cidr
  map_public_ip_on_launch = true
  availability_zone       = "${var.region}a"
  tags = {
    Name = "public-subnet"
  }
}

# Private Subnets
resource "aws_subnet" "private_subnets" {
  count                   = 6
  vpc_id                  = aws_vpc.main_vpc.id
  cidr_block              = var.private_subnet_cidrs[count.index]
  map_public_ip_on_launch = false
  availability_zone       = "${var.region}${element(["a", "b", "c"],
count.index % 3)}"
  tags = {
    Name = "private-subnet-${count.index + 1}"
  }
}

# Route Table (Public)
resource "aws_route_table" "public_rt" {
  vpc_id = aws_vpc.main_vpc.id
  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.gw.id
```

```
  }
}

resource "aws_route_table_association" "public_association" {
  subnet_id      = aws_subnet.public_subnet.id
  route_table_id = aws_route_table.public_rt.id
}

# Security Group - Allow SSH and HTTP
resource "aws_security_group" "web_sg" {
  name        = "web-sg"
  description = "Allow SSH and HTTP"
  vpc_id      = aws_vpc.main_vpc.id

  ingress {
    description = "SSH"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    description = "HTTP"
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port   = 0
```

```
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = {
    Name = "web-sg"
  }
}

# EC2 Instance
resource "aws_instance" "web_server" {
  ami           = var.ami_id
  instance_type = var.instance_type
  subnet_id     = aws_subnet.public_subnet.id
  vpc_security_group_ids = [aws_security_group.web_sg.id]

  user_data = file("userdata.sh")

  tags = {
    Name = "akbar-web-instance"
  }
}
```

✅ Steps to Deploy

1. Save all files in a folder `cloud-vpc-project/`
2. Open terminal in that folder

3. Run:

bash
CopyEdit
```
terraform init
terraform plan
terraform apply -auto-approve
```

✅ What This Deploys

- A full **VPC** with **7 subnets** (6 private, 1 public)
- **Internet Gateway + Route Table** for public access
- **EC2** instance in **public subnet** with **Apache (httpd)** running
- **2 inbound rules** in SG: **SSH (22)** and **HTTP (80)**
- User can access the EC2 **public IP in browser** and see a webpage